

DSA & Advanced DSA Roadmap

Phase 13 — Trie

Learning Objectives

By the end of this phase, learners will be able to:

- Implement a trie (prefix tree) from scratch supporting insert, exact-word search, and prefix search.
- Explain why a trie gives $O(L)$ lookup (L = word length) regardless of how many words are stored, unlike a hash set which is $O(L)$ per lookup but offers no native prefix support.
- Recognize when a trie is the right structure: any problem centered on shared prefixes, autocomplete, or repeated prefix queries over a fixed dictionary.
- Combine a trie with DFS/backtracking (Phase 10) to search a 2D grid for multiple words efficiently.
- Solve Easy to Medium level trie problems independently, and approach Hard problems (Word Search II) by combining trie-based pruning with grid backtracking.

Implementation & Internals

Core Concepts

- Trie Node Design — each node holds a fixed-size array (or hash map) of children, one slot per possible next character, plus a boolean flag marking whether a complete word ends at this node.
- Insert — walking character by character from the root, creating a new child node whenever the next character's path doesn't yet exist, then marking the final node as a word ending.
- Search — walking the same way, but failing immediately if a character's path is missing, and requiring the final node's word-ending flag to be true (not just that the path exists).
- Prefix Search — identical traversal to search, but without requiring the final node to be a word ending — any reachable node confirms the prefix exists in some stored word.

Implementation Exercises

- Build a trie from scratch using an array of 26 children per node (lowercase English letters), implementing insert, search, and startsWith.
- Rebuild the same trie using a hash map of children per node instead of a fixed array, and discuss the space/time trade-off for sparse alphabets (e.g. Unicode) versus the fixed-size approach.
- Extend the basic trie node with a count field, and use it to answer 'how many inserted words have this exact prefix' in $O(L)$ time.

Pattern Mastery

Pattern 1: Auto Complete

Concepts

- Storing a dictionary of words in a trie so that, given a typed prefix, all matching words can be found by walking to the prefix's node and then exploring every word-ending node beneath it.
- Supporting incremental search-as-you-type by maintaining the current trie node pointer across calls, rather than re-walking from the root for every keystroke.

Practice Problems

- Design a search-autocomplete-style data structure supporting word insertion and exact-match search, plus prefix existence checks.
- Given a dictionary of word roots and a sentence, replace every word in the sentence with the shortest root that is a prefix of it, if one exists.

LeetCode

- [Implement Trie \(Prefix Tree\) — leetcode.com/problems/implement-trie-prefix-tree](https://leetcode.com/problems/implement-trie-prefix-tree)
- [Replace Words — leetcode.com/problems/replace-words](https://leetcode.com/problems/replace-words)

Pattern 2: Dictionary Search

Concepts

- Using a trie to support wildcard or pattern-based search over a stored dictionary, exploring multiple branches when a wildcard character could match any letter.
- Verifying not just that a word exists, but that every prefix of it independently exists as a complete word — a stronger condition than ordinary trie search.

Practice Problems

- Design a data structure that supports adding words and searching for a word that may contain the wildcard character '.', matching any single letter.
- Given a list of words, find the longest word that can be built one character at a time using other words already in the list.

LeetCode

- [Design Add and Search Words Data Structure — leetcode.com/problems/design-add-and-search-words-data-structure](https://leetcode.com/problems/design-add-and-search-words-data-structure)
- [Longest Word in Dictionary — leetcode.com/problems/longest-word-in-dictionary](https://leetcode.com/problems/longest-word-in-dictionary)

Pattern 3: Prefix Matching

Concepts

- Combining a trie with grid backtracking (Phase 10): instead of searching the grid once per target word, build one trie from all target words and let the grid DFS prune any path whose current prefix doesn't exist in the trie.
- Marking trie nodes as 'already found' after a match to avoid reporting the same word twice if multiple grid paths reach it.

Practice Problems

- Given an $m \times n$ board of characters and a list of words, find all words that can be formed by sequences of adjacent letters on the board, using a single shared trie to prune the search across all words simultaneously.

LeetCode

- [Word Search II — leetcode.com/problems/word-search-ii](https://leetcode.com/problems/word-search-ii)

Phase Assessment

Implementation Tasks

- Build a trie from scratch supporting insert, search, and startsWith, backed by a fixed-size children array.
- Extend the trie to support wildcard search (matching any single character), reusing the DFS-with-branching technique from Phase 10's backtracking template.
- Analyze and explain why Word Search II's combined trie-plus-grid-DFS approach is faster than running a separate grid search for each word independently.

Recommended LeetCode Target

Medium Problems

Auto Complete

- leetcode.com/problems/implement-trie-prefix-tree
- leetcode.com/problems/replace-words

Dictionary Search

- leetcode.com/problems/design-add-and-search-words-data-structure
- leetcode.com/problems/longest-word-in-dictionary

Hard Problems

- leetcode.com/problems/word-search-ii

On scope and dedup

This phase has 5 distinct problems and no Easy tier, similar in shape to Phase 10 (Backtracking): trie problems are almost always at least Medium difficulty, since the data structure's value only becomes apparent once a problem has enough words or queries to make the prefix-sharing worthwhile. None of the 5 problems overlaps with any problem used in Phases 0–12.

Coding Challenges (Assessment Day)

- Implement Trie (Prefix Tree)
- Design Add and Search Words Data Structure
- Replace Words
- Word Search II

Expected Outcome

Students should be able to:

- Implement a trie from scratch and explain its time and space trade-offs against a hash set for prefix-based queries.
- Recognize and apply the three trie patterns covered: autocomplete, dictionary search with wildcards, and prefix-matching grid search.
- Solve unseen trie problems using pattern-based thinking rather than memorized solutions.
- Combine a trie with techniques from earlier phases (backtracking, grid DFS) to solve compound Hard problems.